

LOME: A Low-latency Optimal Ports Matching Engine for Optical Circuit Switches

Yangmiao Yu^(1,2), Can Chen⁽¹⁾, Yifan Zhang⁽¹⁾, Dawei Zang^{(1,2,3)*}

⁽¹⁾ Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China zangdawei@ict.ac.cn

⁽²⁾ University of Chinese Academy of Sciences, Beijing, China

⁽³⁾ State Key Lab of Processors, ICT, CAS, Beijing, China

*Corresponding author. All authors contributed equally to this manuscript.

Abstract We propose a hardware-accelerated, low-latency, optimal ports matching engine for optical circuit switches to accommodate mouse flows. It reduces port-matching latency to hundreds of nanoseconds, an improvement by multiple orders of magnitude compared to CPU baselines, and reduces mouse flow latency by a factor of 19 on the demo. ©2026 The Author(s)

Introduction

Emerging AI workloads, particularly large language models, are driving explosive traffic growth in modern data centers [1]. The massive data exchange required for training and inference poses significant demands on network bandwidth and latency. In contrast to conventional electronic packet switching, which suffers from limited bandwidth and scalability, optical circuit switching (OCS) establishes dedicated end-to-end optical connections, enabling ultra-high-bandwidth transmission with minimal latency. As a key technology to support intensive communication requirements, OCS is increasingly adopted in large-scale data centers [2, 3].

A critical challenge in managing OCS optical paths is efficiently matching the input and output ports of the optical switch to maximize data throughput. As shown in Fig. 1(a-b), the port matching task is typically formulated as a maximum bipartite matching problem, which can be solved in polynomial time using optimization algorithms such as the Auction Algorithm [4, 5] and the Hungarian Algorithm [6]. Due to the high computational complexity, the time required to com-

pute an optimal match on CPU devices can reach several tens of milliseconds. Limited by scheduling latency, optical switch fabrics can only serve network flows with large amounts of data, called elephant flows, resulting in low optical bandwidth utilization. Recent advancements in photonic integration technology have enabled optical switches based on integrated optical waveguides to achieve switching speeds ranging from nanoseconds to microseconds [7], surpassing the speeds of previous low-speed devices. However, due to limitations in CPU performance, the computational latency of the optimal port matching algorithm lags significantly behind that of emerging optical switching devices. This discrepancy between scheduling time and switching time results in underutilized bandwidth and increased end-to-end packet latency, thereby limiting the adaptability of OCS devices to a broader range of network applications. The performance of optical switch fabrics is bottlenecked by the schedulers' latency.

In this paper, we aim to enable the OCS fabric to handle mouse traffic with low latency by reducing scheduling time. Hence, we propose LOME, a novel hardware-accelerated low-latency schedul-

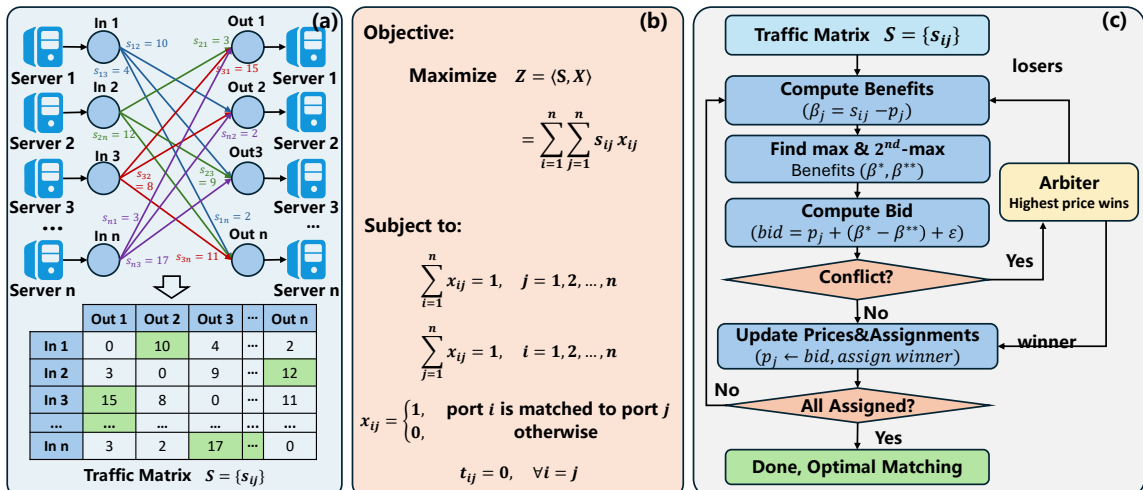


Fig. 1: (a) Link requests from servers form an $n \times n$ traffic matrix $S = \{s_{ij}\}$. (b) Ports matching task is formulated as a maximum weight bipartite matching problem. (c) The primary process of Auction algorithm.

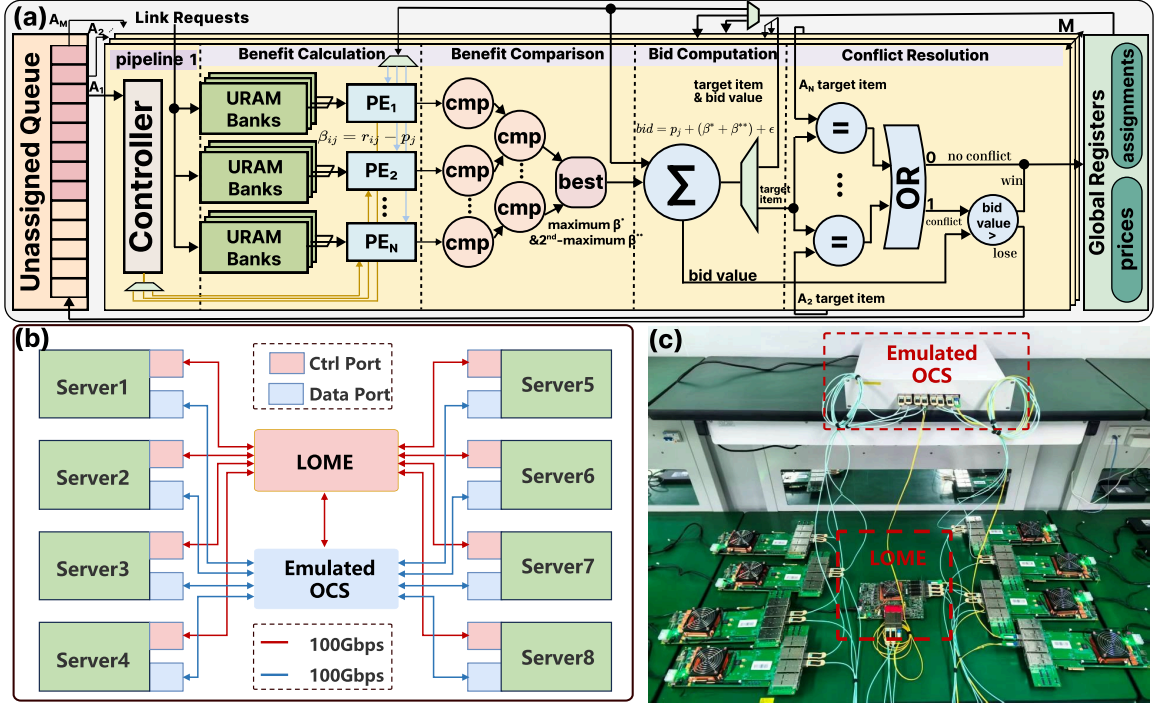


Fig. 2: (a) LOME Architecture. (b) Diagram of the 8-node prototype. (c) Photo of the 8-node prototype.

ing engine for optimal port matching tasks. In the engine, a novel parallel pipeline architecture tailored to the computational features of the Auction Algorithm is designed to significantly reduce port matching computation time. A LOME prototype is implemented on the AMD VU37P FPGA [8] and evaluated for switch radix from 8 to 512 ports. The experimental results demonstrate that the latency of port matching computation can be reduced to hundreds of nanoseconds, an improvement by multiple orders of magnitude compared to CPU implementation. Also, a demo with 8 nodes is designed to mimic rack-scale computing systems, and the average latency of mouse flow with 1 KB data can be reduced to 1/19th that of conventional CPU-based implementations.

Methods and Architecture

The Auction algorithm [5] models the assignment problem as a bidding process where 'buyers' (servers) compete for 'items' (optical links) by placing bids. It works iteratively, where each buyer updates their bid based on the prices of the items they are interested in, gradually converging towards an optimal assignment. As shown in Fig. 1(c), the primary processes involve each server bidding on optical links, adjusting prices based on feedback, and eventually converging to an assignment that maximizes the overall bandwidth. Naturally, the Auction algorithm and the Hungarian method [6] have the same computational complexity and similar CPU latency. However, key features of the Auction algorithm include its distributed nature, which allows it to be potentially parallelized and accelerated by FPGAs or

ASICs. There are two challenges in parallelizing the Auction algorithm: one is streamlining the datapath across multiple processes, and the other is ensuring item price consistency to avoid conflicts when multiple buyers target the same resource simultaneously.

For the first challenge, we map each algorithm process to a dedicated pipeline stage and assign tailored parallelism to each stage to tackle load imbalance across the algorithm processes. This allows the entire datapath to be fully pipelined at a one-bid-per-cycle throughput, reducing latency while maximizing hardware resource efficiency. As illustrated in Fig. 2(a), each pipeline of LOME comprises four stages, including benefit calculation, benefit comparison, bid computation, and conflict resolution. In the benefit calculation stage, the traffic matrix is stored in URAM with 8 consecutive 9-bit values packed into a 72-bit word, and the N elements of each row are cyclically distributed across $N/8$ URAMs, so that N values can be read out in a single cycle to N processing elements (PEs) and then compute the benefits $\beta_j = s_{ij} - p_j$ in parallel. In the benefit comparison stage, these N benefits flow into a binary comparator tree that yields the maximum benefit β^* with its item j^* and the second-maximum benefit β^{**} . In the bid computation stage, the output of the previous stage is used to compute the bid $b = p_{j^*} + (\beta^* - \beta^{**}) + \epsilon$ for its preferred item j^* , and the resulting (target item, bid value) pair is forwarded to the conflict resolution stage. In the conflict resolution stage, a portion of a distributed arbiter is used to resolve contention among pipelines. To make the latest

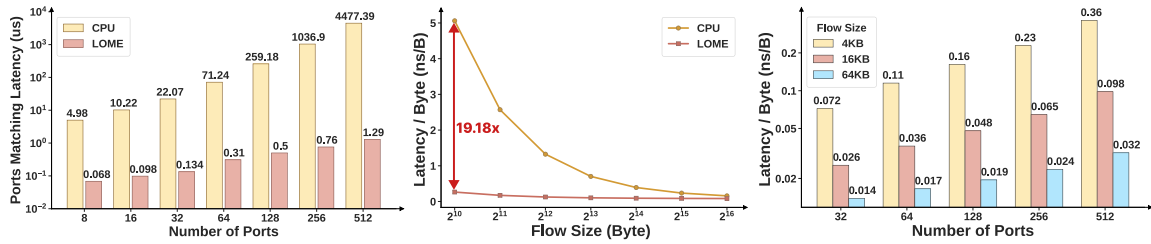


Fig. 3: (a) Ports matching latency. (b) Per-byte latency on the 8-ports prototype. (c) Per-byte latency at 32–512 ports.

prices p_j and assignments visible to every stage with single-cycle access, we keep them in a set of on-chip global registers that any pipeline stage can read or update within the same cycle.

For the second challenge, we implement a system-level arbiter in the conflict resolution stage to resolve price conflicts among different pipelines. In the LOME design, there are M parallel bid pipelines to increase parallelism. The M pipelines share a set of global price and assignment registers to place computation results of each iteration. When multiple pipelines bid on the same item in the same cycle, unresolved collisions would lead to conflicting updates and corrupt the shared state. To prevent this, the conflict-resolution logic first identifies pipelines that bid on the same item using equality comparators. It then forwards non-conflicting bids directly and arbitrates conflicting bids by selecting the highest one, so that only the winning update is committed to the global registers. This scheme guarantees atomicity and data integrity, avoids race conditions, and preserves both functional correctness and throughput for the engine.

While LOME is running, the bid tasks in the global unassigned queue are distributed to each pipeline in a round-robin manner. Then, each pipeline performs the computations of its individual stages in parallel under the management of its controller. After arbitration in the conflict resolution stage, successful bidding tasks are committed to the global registers, and unsuccessful ones are written back to the unassigned queue to start a new iteration. The engine terminates when no unassigned task submits a valid bid, or when the bidding process converges such that no further price updates occur across all items. At this point, an optimal port matching result is achieved, and the engine exits the iterative bidding loop.

Experimental Results

To evaluate the proposed architecture, we implement a prototype on an AMD VU37P FPGA. Firstly, we test LOME computation latency across switch radixes from 8 to 512. As a baseline, we run the implementation of the Hungarian algorithm from the SciPy library [9], which is commonly used in prior work [10, 11], on an AMD Ryzen 9950X CPU. The traffic matrix is randomly generated with values ranging from 0 to 255, and the results are

averaged over 500 tests. As illustrated in Fig. 3(a), LOME achieves the latency of 68 ns on an 8-port switch, and 1.29 μ s on a 512-port switch with a speedup of up to 3000 $\times$. Compared with our previous work, the latency is further reduced by 50 $\times$ [12]. To the best of our knowledge, this is the best performance among existing works.

As shown in Fig. 2(b, c), we also build a demo with 8 nodes to mimic a rack-scale computing system. In the demo, FPGA boards are used to emulate the behavior of servers and OCS device. The switching latency of this emulated OCS device is approximately 10 ns. The servers connect to LOME and the emulated OCS separately using 100 Gb/s optical links. Leveraging an extremely lightweight protocol, the point-to-point transmission latency between LOME and the servers is about 65 ns. As illustrated in Fig. 3(b), LOME is more efficient in handling mouse flows, and the average latency of mouse flow with 1 KB data can be reduced to 1/19th that of conventional CPU-based implementations.

Moreover, we evaluate the mouse-flow latency of various switch radixes with simulation. The bandwidth of the optical link between the server and the OCS device is assumed to be 800 Gb/s. As illustrated in Fig. 3(c), the latency of mouse flow exhibits a slow increase with porting number scaling, and it is still very low for a 512-port device.

Conclusions

In this paper, we design a hardware-accelerated, low-latency, optimal ports matching engine for OCS fabric to accommodate mouse flows. The port-matching latency improves by multiple orders of magnitude compared to CPU baselines. Experimental results show LOME can reduce the average flow latency by a factor of 19 on an 8-node demo. Future work includes using LOME in scale-up scenarios to speed up the XPU system.

Acknowledgements

This work is partially supported by the National Key Research and Development Program of China (2021YFF0704300), the Youth Innovation Promotion Association CAS, the Young Scientists Fund of the National Natural Science Foundation of China (Grant No. 61902373), and the Foundation of the Chinese Academy of Sciences, China (Grant No. JCPYJJ22013).

References

- [1] W. Wang et al., "TopoOpt: Co-optimizing network topology and parallelization strategy for distributed training jobs", in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, Boston, MA: USENIX Association, Apr. 2023, pp. 739–767, ISBN: 978-1-939133-33-5.
- [2] R. Urata et al., "Mission apollo: Landing optical circuit switching at datacenter scale", *arXiv preprint arXiv:2208.10041*, 2022.
- [3] H. Ballani et al., "Sirius: A flat datacenter network with nanosecond optical switching", in *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, 2020, pp. 782–797.
- [4] D. P. Bertsekas, "Auction algorithms for network flow problems: A tutorial introduction", *Computational optimization and applications*, vol. 1, no. 1, pp. 7–66, 1992.
- [5] D. P. Bertsekas, "The auction algorithm: A distributed relaxation method for the assignment problem", *Annals of operations research*, vol. 14, no. 1, pp. 105–123, 1988.
- [6] R. Jonker and A. Volgenant, "A shortest augmenting path algorithm for dense and sparse linear assignment problems", *Computing*, vol. 38, no. 4, pp. 325–340, 1987.
- [7] R. Konoike, K. Suzuki, and K. Ikeda, "Path-independent insertion loss 8×8 silicon photonics switch with nanosecond-order switching time", *Journal of Lightwave Technology*, vol. 41, no. 3, pp. 865–870, 2023.
- [8] AMD. "Virtex UltraScale+ FPGAs product tables and product selection guide". [Online]. Available: <https://www.amd.com/en/products/adaptive-socs-and-fpgas/fpga/virtex-ultrascale-plus.html>
- [9] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python", *Nature methods*, vol. 17, no. 3, pp. 261–272, 2020.
- [10] Y. Mao et al., *ATRO: A fast algorithm for topology engineering of reconfigurable datacenter networks*, 2025. arXiv: 2507.13717 [cs.NI]. [Online]. Available: <https://arxiv.org/abs/2507.13717>
- [11] K. Liang, L. Qiao, I. Keslassy, and B. Lin, "Scheduling parallel optical switches in datacenters", *IEEE Networking Letters*, 2026.
- [12] Y. Yu, Y. Song, Y. Zhang, and D. Zang, "HOCSS: A hardware-accelerated optical circuit switch scheduler for low-latency optimal ports matching", *Optical Fiber Communication Conference and Exposition*, 2026.